

LAPORAN PRAKTIKUM IF12D1
PEMROGRAMAN BERORIENTASI OBJEK
TUGAS PERTEMUAN 6

Disusun untuk Memenuhi Tugas Mata Kuliah Pemrograman Berorientasi Objek

Dosen Pengampu:

Mutaqin Akbar, S.Kom., M.T., MCE., MCF.



Disusun oleh :

Nama : DIMAS NURDIANSYAH
NIM : 251110058

UNIVERSITAS MERCU BUANA YOGYAKARTA
FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI INFORMATIKA
2025/2026

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBAR	iii
A. Tugas	1
B. Alat dan Bahan	1
C. Jawaban.....	2
1. Rancangan Program	2
2. Source Code	3
2.1 Class MainCharacter.java	3
2.2 Class Serangan.java	4
2.3 Class SeranganFisik.java	5
2.4 Class SeranganSihir.java	6
2.5 Class SenjataPanah.java	7
2.6 Class SihirEs.java	8
2.7 Class GameMain.java	9
3. Output.....	10
4. Kesimpulan.....	12

DAFTAR GAMBAR

Gambar 2. 1 Class MainCharacter.java.....	3
Gambar 2. 2 Class Serangan.java	4
Gambar 2. 3 Class SeranganFisik.java	5
Gambar 2. 4 Class SeranganSihir.java	6
Gambar 2. 5 Class SenjataPanah.java.....	7
Gambar 2. 6 Class SihirEs.java.....	8
Gambar 2. 7 Class GameMain.java	9
Gambar 3. 1 Output Program.....	10

A. Tugas

Suatu hari, di kelas Praktikum PBO 12D1 dan 12D2, tiba-tiba terpancar cahaya yang menyilaukan mata, sampai-sampai kamu tidak bisa melihat. Setelah kamu membuka mata, kamu tersadar, kamu dipindah secara langsung ke dunia lain yang menjunjung tinggi Ilmu Pedang dan Sihir. Kamu mendapat kesempatan untuk menjadi Hero atau DemonKing. (Nim Ganjil menjadi Hero, Nim Genap menjadi DemonKing). Kamu diperintah dengan keras oleh masing-masing kerajaan (Kerajaan Manusia/Kerajaan Abyss) untuk melawan masing-masing kerajaan tersebut.

Buatlah sebuah kelas utama (main) sebagai eksekutor kode kalian, superclass MainCharacter sebagai identitas kalian, superclass Serangan sebagai indikator serangan kalian, dan subclass SeranganFisik, SeranganSihir sebagai serangan kalian ke kubu yang kalian lawan.

- Untuk kubu Hero, senjata kalian adalah Pedang Suci untuk SeranganFisik, dan Sihir Cahaya untuk SeranganSihir.
- Untuk kubu DemonKing, senjata kalian adalah Pedang Kutukan untuk SeranganFisik, dan Sihir Hitam untuk SeranganSihir.

B. Alat dan Bahan

1. Komputer
2. Modul
3. Java SDK (JDK)
4. Text Editor (VS Code)
5. Command Pemompt (CMD)

C. Jawaban

1. Rancangan Program

Program ini dirancang menggunakan konsep Pemrograman Berorientasi Objek (PBO) dengan menerapkan beberapa konsep utama yaitu Abstraksi, Pewarisan (Inheritance), Polimorfisme (Polymorphism), dan Enkapsulasi (Encapsulation). Program mensimulasikan sistem pertempuran berbasis karakter dalam sebuah dunia fiksi, di mana seorang karakter bernama Demon King memiliki kemampuan untuk melakukan berbagai jenis serangan.

Hierarki kelas dalam program ini tersusun sebagai berikut:

Serangan (Superclass Abstract) → SeranganFisik, SeranganSihir, SenjataPanah, SihirEs (Subclass)

Konsep-konsep PBO yang diterapkan dalam program ini adalah sebagai berikut:

- Abstraksi diterapkan melalui pembuatan kelas abstrak Serangan yang berfungsi sebagai kerangka dasar untuk semua jenis serangan. Kelas ini tidak dapat diinstansiasi secara langsung, melainkan hanya berfungsi sebagai blueprint yang mendefinisikan method abstrak serang() dan efek() yang wajib diimplementasikan oleh seluruh subclass-nya.
- Pewarisan (Inheritance) diterapkan melalui keyword extends pada keempat subclass yaitu SeranganFisik, SeranganSihir, SenjataPanah, dan SihirEs yang semuanya mewarisi kelas Serangan. Dengan pewarisan ini, seluruh subclass secara otomatis memiliki atribut tipeEfek, namaSenjata, dan lastUsed dari superclass tanpa perlu menulis ulang.
- Polimorfisme (Polymorphism) diterapkan melalui mekanisme method overriding, di mana setiap subclass mengimplementasikan method serang() dan efek() dengan perilaku yang berbeda-beda meskipun dipanggil dengan nama method yang sama. Selain itu, polimorfisme juga terlihat dari penggunaan tipe referensi Serangan yang dapat menampung berbagai objek subclass yang berbeda, sehingga satu variabel bertipe Serangan dapat berperilaku sesuai dengan objek yang direferensikannya.
- Enkapsulasi (Encapsulation) diterapkan pada kelas MainCharacter dengan menjadikan atribut nama dan lastSerangan bersifat private, sehingga kedua atribut tersebut hanya dapat diakses dan dimodifikasi melalui method yang telah disediakan, yaitu serang() dan efek().

Alur eksekusi program berjalan dari kelas GameMain sebagai titik masuk, kemudian membuat objek MainCharacter yang langsung mencetak identitas karakter, dilanjutkan dengan pembuatan objek-objek serangan, lalu memanggil method serang() dan efek() secara bergantian hingga seluruh serangan ditampilkan ke layar.

2. Source Code

2.1 Class MainCharacter.java

```
Pertemuan6 > MainCharacter.java > ...
1  package Pertemuan6;
2
3  public class MainCharacter {
4      private String nama;
5      private Serangan lastSerangan;
6
7      public MainCharacter(String nama) {
8          this.nama = nama;
9          System.out.println("Saya " + nama);
10         System.out.println("Saya tergabung di kubu Demon King, untuk menyerang Hero");
11     }
12
13     public void serang(Serangan serangan) {
14         this.lastSerangan = serangan;
15         serangan.serang(this.nama);
16     }
17
18     public void efek() {
19         if (this.lastSerangan != null) {
20             this.lastSerangan.efeK(this.nama);
21         }
22     }
23 }
```

Gambar 2. 1 Class MainCharacter.java

Penjelasan Kode Program :

Class MainCharacter berfungsi sebagai kelas entitas karakter utama yang merepresentasikan identitas pemain dalam program ini. Kelas ini menerapkan konsep enkapsulasi dan menjadi penghubung antara karakter dengan berbagai jenis serangan yang dimilikinya.

- `private String nama` → atribut untuk menyimpan nama karakter pemain. Menggunakan akses modifier `private` sebagai penerapan enkapsulasi sehingga nilai atribut ini hanya dapat diakses melalui method yang telah disediakan.
- `private Serangan lastSerangan` → atribut untuk menyimpan referensi objek serangan terakhir yang digunakan. Bertipe Serangan (superclass) sehingga dapat menampung objek dari subclass manapun — inilah penerapan polimorfisme pada level atribut.
- `public MainCharacter(String nama)` → konstruktor yang menginisialisasi nama karakter dan langsung mencetak identitas karakter beserta keterangan kubu ke layar saat objek dibuat.
- `public void serang(Serangan serangan)` → method yang menerima objek bertipe Serangan sebagai parameter, menyimpannya ke `lastSerangan`, lalu mendelegasikan eksekusi serangan ke objek tersebut melalui `serangan.serang(this.nama)`. Karena parameternya bertipe superclass Serangan, method ini dapat menerima objek dari keempat subclass yang berbeda — inilah inti dari penerapan polimorfisme dalam program ini.
- `public void efek()` → method yang memanggil `efeK()` dari objek serangan terakhir yang tersimpan di `lastSerangan`. Terdapat pengecekan `null` untuk memastikan program tidak crash jika `efeK()` dipanggil sebelum `serang()`.

2.2 Class Serangan.java

```
Pertemuan6 > Serangan.java > ...
1  package Pertemuan6;
2
3  public abstract class Serangan {
4      protected String tipeEfek;
5      protected String namaSenjata;
6      protected boolean lastUsed;
7
8      public abstract void serang(String namaPengguna);
9      public abstract void efek(String namaPengguna);
10 }
```

Gambar 2. 2 Class Serangan.java

Penjelasan Kode Program :

Class Serangan berfungsi sebagai superclass abstrak yang menjadi fondasi utama dari seluruh jenis serangan dalam program ini. Kelas ini tidak dapat diinstansiasi secara langsung karena bersifat abstract, melainkan hanya digunakan sebagai kerangka dasar yang wajib diimplementasikan oleh subclass-nya.

- `public abstract class Serangan` → mendeklarasikan bahwa kelas ini adalah kelas abstrak yang tidak dapat dibuat objeknya secara langsung. Semua kelas serangan wajib mewarisi kelas ini.
- `protected String tipeEfek` → atribut untuk menyimpan jenis efek kerusakan yang dihasilkan oleh serangan, seperti Physical Damage atau Magic Damage. Menggunakan akses modifier `protected` agar dapat diwarisi dan diakses langsung oleh subclass-nya.
- `protected String namaSenjata` → atribut untuk menyimpan nama senjata atau sihir yang sedang digunakan saat method `serang()` dipanggil.
- `protected boolean lastUsed` → atribut penanda apakah serangan ini baru saja digunakan, dimanfaatkan untuk keperluan logika internal subclass.
- `public abstract void serang(String namaPengguna)` → deklarasi method abstrak yang wajib di-override oleh seluruh subclass. Method ini bertanggung jawab untuk mencetak aksi penggunaan senjata ke layar.
- `public abstract void efek(String namaPengguna)` → deklarasi method abstrak yang wajib di-override oleh seluruh subclass. Method ini bertanggung jawab untuk mencetak jenis efek kerusakan yang dihasilkan.

2.3 Class SeranganFisik.java

```
Pertemuan6 > SeranganFisik.java > ...
1  package Pertemuan6;
2
3  public class SeranganFisik extends Serangan {
4
5      private String[] daftarSenjata = { "Pedang Kutukan" };
6      private int indexSenjata = 0;
7
8      public SeranganFisik() {
9          this.tipeEfek = "Physical Damage";
10         System.out.println("Saya memiliki senjata berupa Pedang Kutukan");
11     }
12
13     @Override
14     public void serang(String namaPengguna) {
15         this.namaSenjata = daftarSenjata[indexSenjata % daftarSenjata.length];
16         indexSenjata++;
17         this.lastUsed = true;
18         System.out.println(namaPengguna + " menggunakan " + namaSenjata);
19     }
20
21     @Override
22     public void efek(String namaPengguna) {
23         this.lastUsed = false;
24         System.out.println(namaPengguna + " memberikan efek " + tipeEfek);
25     }
26 }
27
```

Gambar 2. 3 Class SeranganFisik.java

Penjelasan Kode Program :

Class SeranganFisik adalah subclass pertama yang mewarisi kelas abstrak Serangan menggunakan keyword extends. Kelas ini merepresentasikan serangan fisik wajib menggunakan Pedang Kutukan yang dimiliki oleh Demon King.

- `public class SeranganFisik extends Serangan` → mendeklarasikan bahwa kelas ini mewarisi seluruh atribut dan method abstrak dari kelas Serangan, sehingga wajib mengimplementasikan method `serang()` dan `efek()`.
- `private String[] daftarSenjata` → array untuk menyimpan daftar nama senjata fisik yang tersedia. Menggunakan array agar program dapat dikembangkan dengan menambahkan lebih banyak senjata tanpa mengubah logika utama.
- `private int indexSenjata` → variabel penunjuk index array yang bertambah setiap kali method `serang()` dipanggil, memungkinkan rotasi senjata jika terdapat lebih dari satu senjata dalam array.
- `public SeranganFisik()` → konstruktor yang menginisialisasi `tipeEfek` dengan nilai "Physical Damage" dan langsung mencetak informasi kepemilikan senjata ke layar saat objek dibuat.
- `@Override public void serang(String namaPengguna)` → implementasi dari method abstrak `serang()`. Method ini mengambil nama senjata dari array berdasarkan index, menaikkan nilai index, menandai `lastUsed` menjadi true, lalu mencetak aksi penggunaan senjata ke layar.
- `@Override public void efek(String namaPengguna)` → implementasi dari method abstrak `efek()`. Method ini mereset `lastUsed` menjadi false lalu mencetak jenis efek kerusakan yang dihasilkan oleh serangan fisik.

2.4 Class SeranganSihir.java

```
Pertemuan6 > SeranganSihir.java > ...
1  package Pertemuan6;
2
3  public class SeranganSihir extends Serangan {
4
5      private String[] daftarSihir = { "Sihir Hitam", "Sihir Api Neraka" };
6      private int indexSihir = 0;
7
8      public SeranganSihir() {
9          this.tipeEfek = "Magic Damage";
10         System.out.println("Saya memiliki sihir berupa Sihir Hitam");
11     }
12
13     @Override
14     public void serang(String namaPengguna) {
15         this.namaSenjata = daftarSihir[indexSihir % daftarSihir.length];
16         indexSihir++;
17         this.lastUsed = true;
18         System.out.println(namaPengguna + " menggunakan " + namaSenjata);
19     }
20
21     @Override
22     public void efek(String namaPengguna) {
23         this.lastUsed = false;
24         System.out.println(namaPengguna + " memberikan efek " + tipeEfek);
25     }
26 }
```

Gambar 2. 4 Class SeranganSihir.java

Penjelasan Kode Program :

Class SeranganSihir adalah subclass kedua yang mewarisi kelas abstrak Serangan. Kelas ini merepresentasikan serangan sihir wajib menggunakan Sihir Hitam, dengan tambahan Sihir Api Neraka sebagai sihir tambahan kreatif.

- `public class SeranganSihir extends Serangan` → mendeklarasikan pewarisan dari kelas Serangan, sehingga wajib mengimplementasikan seluruh method abstraknya.
- `private String[] daftarSihir` → array yang menyimpan dua sihir sekaligus, yaitu "Sihir Hitam" sebagai sihir wajib sesuai ketentuan soal dan "Sihir Api Neraka" sebagai sihir tambahan kreatif. Penggunaan array ini juga mendemonstrasikan bahwa satu class dapat menampung lebih dari satu variasi serangan.
- `private int indexSihir` → variabel penunjuk index untuk mekanisme rotasi sihir dalam array, bertambah setiap kali method `serang()` dipanggil.
- `public SeranganSihir()` → konstruktor yang menginisialisasi `tipeEfek` dengan nilai "Magic Damage" dan mencetak informasi kepemilikan sihir ke layar.
- `@Override public void serang(String namaPengguna)` → implementasi method abstrak yang mengambil nama sihir dari array, menaikkan index, lalu mencetak aksi penggunaan sihir.
- `@Override public void efek(String namaPengguna)` → implementasi method abstrak yang mencetak jenis efek kerusakan sihir ke layar.

2.5 Class SenjataPanas.java

```
Pertemuan6 > SenjataPanas.java > ...
1  package Pertemuan6;
2
3  public class SenjataPanas extends Serangan {
4
5      private String[] daftarSenjata = { "Panah Beracun", "Panah Petir" };
6      private int indexSenjata = 0;
7
8      public SenjataPanas() {
9          this.tipeEfek = "Physical Damage + Poison";
10         System.out.println("Saya memiliki senjata berupa Panah Beracun");
11     }
12
13     @Override
14     public void serang(String namaPengguna) {
15         this.namaSenjata = daftarSenjata[indexSenjata % daftarSenjata.length];
16         indexSenjata++;
17         this.lastUsed = true;
18         System.out.println(namaPengguna + " menembakkan " + namaSenjata);
19     }
20
21     @Override
22     public void efek(String namaPengguna) {
23         this.lastUsed = false;
24         System.out.println(namaPengguna + " memberikan efek " + tipeEfek);
25     }
26 }
```

Gambar 2. 5 Class SenjataPanas.java

Penjelasan Kode Program :

Class SenjataPanas adalah subclass ketiga yang mewarisi kelas abstrak Serangan. Kelas ini merupakan senjata tambahan kreatif berupa serangan jarak jauh menggunakan panah yang memberikan efek racun kepada musuh.

- `public class SenjataPanas extends Serangan` → mendeklarasikan pewarisan dari kelas Serangan dan wajib mengimplementasikan method abstraknya.
- `private String[] daftarSenjata` → array yang menyimpan dua varian panah yaitu "Panah Beracun" dan "Panah Petir" sebagai pilihan serangan jarak jauh.
- `public SenjataPanas()` → konstruktor yang menginisialisasi `tipeEfek` dengan nilai "Physical Damage + Poison", menunjukkan bahwa serangan ini memberikan dua efek sekaligus yaitu kerusakan fisik dan racun.
- `@Override public void serang(String namaPengguna)` → implementasi method abstrak dengan kata kerja "menembakkan" yang berbeda dari subclass lain, menunjukkan karakteristik unik serangan jarak jauh ini sebagai salah satu bentuk polimorfisme.
- `@Override public void efek(String namaPengguna)` → implementasi method abstrak yang mencetak efek gabungan Physical Damage dan Poison ke layar.

2.6 Class SihirEs.java

```
Pertemuan6 > SihirEs.java > ...
1  package Pertemuan6;
2
3  public class SihirEs extends Serangan {
4
5      private String[] daftarSihir = { "Tombak Es", "Badai Salju" };
6      private int indexSihir = 0;
7
8      public SihirEs() {
9          this.tipeEfek = "Magic Damage + Freeze";
10         System.out.println("Saya memiliki sihir berupa Tombak Es");
11     }
12
13     @Override
14     public void serang(String namaPegguna) {
15         this.namaSenjata = daftarSihir[indexSihir % daftarSihir.length];
16         indexSihir++;
17         this.lastUsed = true;
18         System.out.println(namaPegguna + " merapalkan " + namaSenjata);
19     }
20
21     @Override
22     public void efek(String namaPegguna) {
23         this.lastUsed = false;
24         System.out.println(namaPegguna + " memberikan efek " + tipeEfek);
25     }
26 }
```

Gambar 2. 6 Class SihirEs.java

Penjelasan Kode Program :

Class SihirEs adalah subclass keempat yang mewarisi kelas abstrak Serangan. Kelas ini merupakan sihir tambahan kreatif berupa serangan es yang mampu membekukan musuh.

- `public class SihirEs extends Serangan` → mendeklarasikan pewarisan dari kelas Serangan dan wajib mengimplementasikan seluruh method abstraknya.
- `private String[] daftarSihir` → array yang menyimpan dua varian sihir es yaitu "Tombak Es" dan "Badai Salju".
- `public SihirEs()` → konstruktor yang menginisialisasi `tipeEfek` dengan nilai "Magic Damage + Freeze", menunjukkan bahwa sihir ini memberikan dua efek sekaligus yaitu kerusakan sihir dan pembekuan.
- `@Override public void serang(String namaPegguna)` → implementasi method abstrak dengan kata kerja "merapalkan" yang unik, berbeda dari subclass lain, semakin mempertegas penerapan polimorfisme dalam program ini.
- `@Override public void efek(String namaPegguna)` → implementasi method abstrak yang mencetak efek gabungan Magic Damage dan Freeze ke layar.

2.7 Class GameMain.java

```
Pertemuan6 > GameMain.java > GameMain
1  package Pertemuan6;
2
3  class GameMain {
4      public static void main(String[] args) {
5          MainCharacter demonKing = new MainCharacter("Dimas");
6
7          System.out.println();
8
9          Serangan pedang = new SeranganFisik();
10         Serangan magic = new SeranganSihir();
11         Serangan panah = new SenjataPanah();
12         Serangan sihirEs = new SihirEs();
13
14         System.out.println();
15
16         demonKing.serang(pedang);
17         demonKing.efek();
18         demonKing.serang(magic);
19         demonKing.efek();
20         demonKing.serang(panah);
21         demonKing.efek();
22         demonKing.serang(sihirEs);
23         demonKing.efek();
24     }
25 }
```

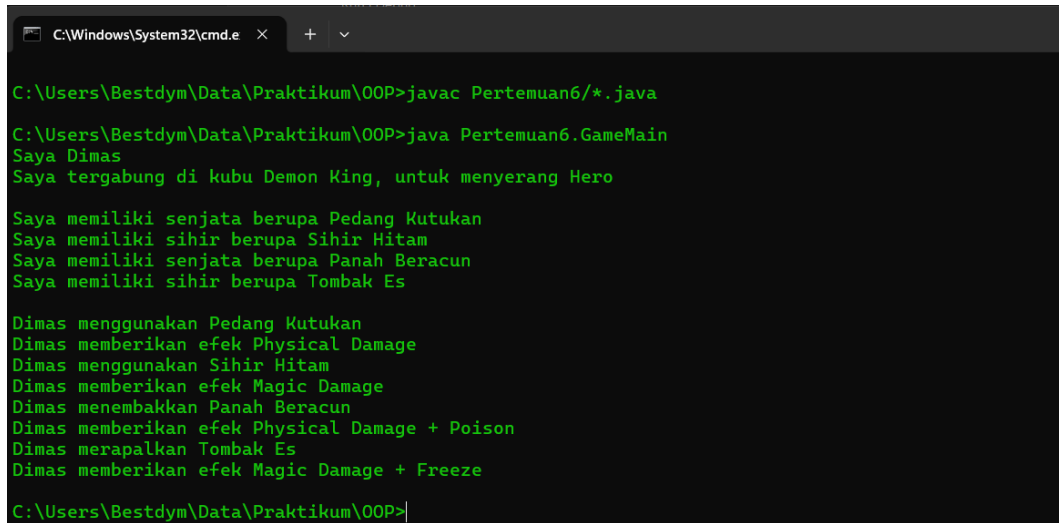
Gambar 2. 7 Class GameMain.java

Penjelasan Kode Program :

Class GameMain berfungsi sebagai entry point (titik masuk) program yang menjalankan seluruh alur eksekusi menggunakan method main.

- `public static void main(String[] args)` → method utama yang secara otomatis dijalankan pertama kali oleh Java Virtual Machine (JVM) saat program dieksekusi.
- `MainCharacter demonKing = new MainCharacter("Dimas")` → membuat objek karakter utama dengan nama "Dimas" yang langsung mencetak identitas dan keterangan kubu ke layar melalui konstruktornya.
- `Serangan pedang = new SeranganFisik()` → membuat objek `SeranganFisik` namun disimpan dalam variabel bertipe `Serangan` (superclass). Inilah penerapan polimorfisme — variabel bertipe superclass dapat menampung objek subclass manapun.
- `Serangan magic = new SeranganSihir()`, `Serangan panah = new SenjataPanah()`, `Serangan sihirEs = new SihirEs()` → hal yang sama diterapkan pada ketiga objek serangan lainnya, seluruhnya bertipe referensi `Serangan` namun masing-masing berperilaku berbeda sesuai subclass-nya.
- `System.out.println()` → digunakan sebagai pemisah antar bagian output agar tampilan lebih rapi dan mudah dibaca.
- `demonKing.serang(pedang)` dan `demonKing.efek()` → pemanggilan method secara berpasangan untuk setiap serangan. Method `serang()` mencetak aksi penggunaan senjata, sedangkan `efek()` mencetak jenis kerusakan yang dihasilkan. Pola ini diulang untuk keempat objek serangan secara berurutan.

3. Output



```
C:\Windows\System32\cmd.e x + v

C:\Users\Bestdym\Data\Praktikum\OOP>javac Pertemuan6/*.java

C:\Users\Bestdym\Data\Praktikum\OOP>java Pertemuan6.GameMain
Saya Dimas
Saya tergabung di kubu Demon King, untuk menyerang Hero

Saya memiliki senjata berupa Pedang Kutukan
Saya memiliki sihir berupa Sihir Hitam
Saya memiliki senjata berupa Panah Beracun
Saya memiliki sihir berupa Tombak Es

Dimas menggunakan Pedang Kutukan
Dimas memberikan efek Physical Damage
Dimas menggunakan Sihir Hitam
Dimas memberikan efek Magic Damage
Dimas menembakkan Panah Beracun
Dimas memberikan efek Physical Damage + Poison
Dimas merapalkan Tombak Es
Dimas memberikan efek Magic Damage + Freeze

C:\Users\Bestdym\Data\Praktikum\OOP>
```

Gambar 3. 1 Output Program

Penjelasan Cara Running :

Untuk menjalankan program, langkah pertama yang dilakukan adalah navigasi, yaitu masuk ke folder tempat kode disimpan melalui CMD menggunakan perintah `cd path\ke\Praktikum\OOP`. Perintah ini wajib dilakukan agar CMD mengetahui lokasi file program yang akan dikompilasi dan dijalankan. Setelah berada di dalam folder yang tepat, langkah selanjutnya adalah kompilasi, yaitu menerjemahkan kode Java menjadi file `.class` yang dapat dimengerti oleh komputer. Terdapat dua cara kompilasi yang dapat dilakukan:

- Cara pertama adalah kompilasi satu per satu, yaitu mengompilasi setiap file secara berurutan mulai dari `Serangan.java`, kemudian `SeranganFisik.java`, `SeranganSihir.java`, `SenjataPanah.java`, `SihirEs.java`, `MainCharacter.java`, dan terakhir `GameMain.java` menggunakan perintah `javac` di setiap filenya. Urutan ini sangat penting karena seluruh subclass bergantung pada superclass `Serangan`, sehingga jika urutannya terbalik maka compiler akan gagal menemukan kelas induknya.
- Cara kedua adalah kompilasi sekaligus menggunakan perintah `javac Pertemuan6/*.java`, di mana tanda `/*.java` berarti mengompilasi semua file `.java` yang ada di dalam folder `Pertemuan6` dalam satu perintah. Cara ini lebih cepat dan efisien karena tidak perlu mengetik nama file satu per satu, namun jika terjadi error maka semua pesan error dari berbagai file akan muncul bersamaan sehingga lebih sulit untuk dilacak.

Setelah proses kompilasi selesai tanpa error dan file `.class` terbentuk untuk setiap kelas, program dieksekusi menggunakan perintah `java Pertemuan6.GameMain`. Perintah ini menjalankan kelas `GameMain` yang di dalamnya terdapat method `main` sebagai titik masuk eksekusi program dan menghasilkan output berupa simulasi pertempuran `Demon King`

yang menampilkan identitas karakter, daftar senjata yang dimiliki, serta aksi serangan beserta efeknya secara terstruktur ke layar.

Penjelasan Output Program :

Output program menampilkan tiga bagian secara berurutan yaitu identitas karakter, daftar senjata yang dimiliki, dan aksi serangan beserta efeknya.

- Bagian pertama adalah identitas karakter yang muncul saat objek MainCharacter dibuat melalui konstruktornya. Program mencetak "Saya Dimas" sebagai nama karakter dan "Saya tergabung di kubu Demon King, untuk menyerang Hero" sebagai keterangan kubu yang diikuti.
- Bagian kedua adalah daftar senjata dan sihir yang muncul saat keempat objek serangan diinstansiasi. Program mencetak "Saya memiliki senjata berupa Pedang Kutukan" saat objek SeranganFisik dibuat, "Saya memiliki sihir berupa Sihir Hitam" saat objek SeranganSihir dibuat, "Saya memiliki senjata berupa Panah Beracun" saat objek SenjataPanah dibuat, dan "Saya memiliki sihir berupa Tombak Es" saat objek SihirEs dibuat. Seluruh output pada bagian ini dihasilkan dari konstruktor masing-masing subclass.
- Bagian ketiga adalah aksi serangan dan efek yang muncul saat method `serang()` dan `efek()` dipanggil secara berpasangan pada objek `demonKing`. Program mencetak delapan baris output secara berurutan:

"Dimas menggunakan Pedang Kutukan" dan "Dimas memberikan efek Physical Damage" → hasil pemanggilan `serang()` dan `efek()` dari objek `SeranganFisik`.

"Dimas menggunakan Sihir Hitam" dan "Dimas memberikan efek Magic Damage" → hasil pemanggilan dari objek `SeranganSihir`.

"Dimas menembakkan Panah Beracun" dan "Dimas memberikan efek Physical Damage + Poison" → hasil pemanggilan dari objek `SenjataPanah`. Perhatikan kata kerja "menembakkan" yang berbeda dari serangan lain, menunjukkan karakteristik unik serangan jarak jauh.

"Dimas merapalkan Tombak Es" dan "Dimas memberikan efek Magic Damage + Freeze" → hasil pemanggilan dari objek `SihirEs`. Kata kerja "merapalkan" yang unik menunjukkan karakteristik serangan sihir.

Perbedaan kata kerja dan efek pada setiap serangan membuktikan bahwa meskipun method yang dipanggil sama yaitu `serang()` dan `efek()`, perilaku yang dihasilkan berbeda-beda sesuai dengan objek subclass yang direferensikan — inilah bukti nyata penerapan konsep Polymorphism dalam program ini.

4. Kesimpulan

Program simulasi pertempuran Demon King ini berhasil diimplementasikan menggunakan konsep-konsep utama Pemrograman Berorientasi Objek (PBO) dalam bahasa pemrograman Java. Hierarki kelas tersusun dengan kelas abstrak Serangan sebagai superclass, dan empat subclass yaitu SeranganFisik, SeranganSihir, SenjataPanah, dan SihirEs sebagai implementasi konkretnya, serta kelas MainCharacter sebagai entitas karakter utama yang menghubungkan semuanya.

Melalui program ini, beberapa konsep penting Pemrograman Berorientasi Objek berhasil diterapkan secara bersamaan. Konsep abstraksi diterapkan dengan menjadikan kelas Serangan sebagai abstract class yang tidak dapat diinstansiasi secara langsung, melainkan hanya berfungsi sebagai kerangka dasar yang memaksa seluruh subclass untuk mengimplementasikan method `serang()` dan `efek()` sesuai karakteristik serangannya masing-masing. Konsep pewarisan (inheritance) diterapkan melalui keyword `extends` pada keempat subclass yang mewarisi seluruh atribut dan method dari superclass Serangan tanpa perlu menulis ulang kode yang sama. Konsep polimorfisme (polymorphism) terlihat secara nyata ketika variabel bertipe Serangan dapat menampung objek dari subclass yang berbeda-beda, dan ketika method `serang()` serta `efek()` dipanggil dengan nama yang sama namun menghasilkan perilaku yang berbeda sesuai objek yang direferensikannya — dibuktikan dari perbedaan kata kerja dan efek pada setiap serangan di output program. Konsep enkapsulasi (encapsulation) diterapkan pada kelas MainCharacter dengan menjadikan atribut `nama` dan `lastSerangan` bersifat `private` sehingga hanya dapat diakses dan dimodifikasi melalui method yang telah disediakan.

Dengan demikian, program ini membuktikan bahwa penerapan konsep-konsep PBO khususnya polimorfisme melalui method overriding dapat membuat kode program menjadi lebih terstruktur, fleksibel, dan mudah dikembangkan, karena penambahan jenis serangan baru cukup dilakukan dengan membuat subclass baru tanpa perlu mengubah kode yang sudah ada sebelumnya.